Reviewer Pack — Minimal Order of Octonion Algebras Bounds Read-Twice BP Size

Entry 2e4b58cddd80 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 05:29:17 UTC

Minimal Order of Octonion Algebras Bounds Read-Twice BP Size

Entry ID: `2e4b58cddd80` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 05:29:17 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Octonion Algebras) **Field B** (complexity object): Boolean Function Complexity (Read-Twice Branching Program Size)

Statement:

> ['For every n-ary Boolean function f, let O(f) be the octonion algebra generated by the Fourier coefficients of f. Then the minimal order of the center of O(f), denoted as ord(C(O(f))), is upper bounded by some polynomial in log(n). Equivalently: for all n-ary boolean functions f, the read-twice BP size BP_rtw(f) satisfies BP_rtw(f) ≤ c * ord(C(O(f))) for some constant c.', 'For all octonion algebras O associated with Boolean functions, there exists a polynomial p(n) such that the minimal order of their centers is upper bounded by p(n).', 'There exists a function h(n) = c * p(n) such that for any boolean function f with n variables, the read-twice BP size BP_rtw(f) ≤ h(n).']

Rationale (proposer's reasoning):

> ['Octonion algebras offer a noncommutative framework to explore properties of Boolean functions beyond classical algebraic structures. Their minimal order may provide insights into the complexity of read-twice branching programs, which are known to be related to the communication complexity of boolean functions.', 'The connection between octonion algebras and read-twice BP size is motivated by the potential for noncommutative algebra to expose new structures that could limit the complexity of computation.', 'Previous work has shown that certain algebraic structures can provide bounds on circuit complexity; this conjecture generalizes such ideas to octonion algebras.']

Taxonomy category: `NoncommutativeAlgebra` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e12d7a221b73ca21`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal order of the center of O(f), ord(C(O(f))), and the read-twice BP size BP_rtw(f) across at least five random seeds exceeds 0.7, with a p-value less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "octonion algebra" AND "read-twice BP size" - "minimal order center" AND "Boolean function complexity" - "Fourier coefficients" AND "branching program size"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def fourier_coefficients(f):
        n = len(f)
        coeffs = []
        for k in range(n + 1):
            coeff = Fraction(0)
            for i in range(2**n):
                term = f[i] * (-1)**sum((i >> j) & 1 for j in range(k))
                coeff += term
            coeff /= 2**n
            coeffs.append(coeff)
        return coeffs

    def construct_octonion_algebra(coeffs):
        n = len(coeffs)
        O = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
```

```
            for j in range(n + 1):
                if i == j:
                    O[i][j] = coeffs[i]
                else:
                    O[i][j] = Fraction(0)
        return O

    def minimal_order_of_center(O):
        n = len(O) - 1
        center = [O[i][i] for i in range(n + 1)]
        order = max(abs(coeff) for coeff in center)
        return int(math.ceil(order))

    def read_twice_bp_size(f):
        n = len(f)
        size = 0
        for i in range(2**n):
            for j in range(i, 2**n):
                if f[i] == f[j]:
                    size += 1
        return size

    results = []
    for _ in range(30):  # Ensure at least 30 instances per seed
        n = random.randint(5, 40)
        f = generate_boolean_function(n)
        coeffs = fourier_coefficients(f)
        O = construct_octonion_algebra(coeffs)
        ord_C_O = minimal_order_of_center(O)
        BP_rtw_f = read_twice_bp_size(f)
        results.append((ord_C_O, BP_rtw_f))

    if not results:
        return {
            "metric_name": "correlation",
            "metric_value": 0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "no_data"
        }

    ord_C_Os, BP_rtw_fs = zip(*results)
    correlation = sum((ord_C_O - mean) * (BP_rtw_f - mean) for ord_C_O, BP_rtw_f in results) / len
    mean = sum(ord_C_Os) / len(ord_C_Os)
    std = math.sqrt(sum((x - mean)**2 for x in ord_C_Os) / len(ord_C_Os))

    return {
        "metric_name": "correlation",
        "metric_value": correlation,
        "instances_tested": len(results),
        "conjecture_holds": abs(correlation) > 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
```

```
seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

if not results:
    print("RESULT: INCONCLUSIVE no_data")
else:
    mean_corr = sum(result["metric_value"] for result in results) / len(results)
    std_corr = math.sqrt(sum((result["metric_value"] - mean_corr)**2 for result in results) /
    support_fraction = sum(1 for result in results if abs(result["metric_value"]) > 0.7) / len

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr} std={std_corr} support_fraction={support_fr
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["c
        print(f"RESULT: FALSIFIED counterexample=\"correlation_threshold_not_met\" first_faili
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the correlation coefficient and p-value could not be computed to assess the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15761
2	preregistration	ollama_remote	glm4:latest	0	0	5552

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	4572
4	novelty	ollama_remote	glm4:latest	0	0	5525
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8988
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7501
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11409
9	judge	ollama_remote	glm4:latest	0	0	23917

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 100608 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2e4b58cddd80.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2e4b58cddd80.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2e4b58cddd80.tar.gz` (if generated)